

AI FOUNDATIONS SERIES · QWIK GUIDE

Reinforcement Learning Tools & Frameworks

A Beginner's Training Guide

Agents, environments, Q-learning, and Deep Q-learning —
explained in plain language, with runnable Python.

Includes a tested companion code package

David Grunwald

1 What this guide covers

Most machine learning you may have met so far is *supervised*: you show the computer labelled examples ("this is a cat, this is a dog") and it learns to copy the labels. **Reinforcement learning (RL)** is different. There are no labels. Instead an **agent** tries things in an **environment**, receives a **reward** when it does well, and gradually learns which actions pay off. It is how you would teach a dog a trick, or how a program learns to play a game.

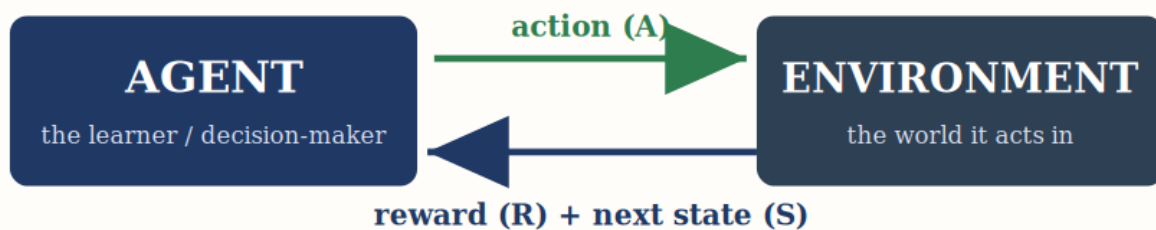
This guide walks a complete beginner from the core vocabulary up to **Q-learning** — the classic RL algorithm — and finally to **Deep Q-learning**, which swaps a lookup table for a small neural network. Every idea is paired with runnable code in the companion package.

HOW TO USE THIS GUIDE

Read a section, then run the matching lesson from the `RL_Sample_Programs` package. Watching the code print its progress is where the ideas truly click.

2 The core idea: the agent-environment loop

Everything in RL is built on one simple loop. The agent looks at the current **state**, picks an **action**, and the environment answers with a **reward** and the **next state**. Then it happens again. And again.



The agent takes an action; the environment answers with a reward and a new state. This loop repeats, and the agent learns to earn more reward over time.

Figure 1 — The agent–environment loop, the heartbeat of all reinforcement learning.

IN PLAIN TERMS

The agent is the *learner*. The environment is the *world*. A *reward* is a point the world hands out for good moves. The agent's whole job is to collect as much reward as it can.

The words you must know first

Term	Everyday meaning
State (S)	Where the agent is right now — its current situation.
Action (A)	A move the agent can make.
Reward (R)	The score the environment returns after a move.
Policy (π)	The agent's strategy — how it picks an action from a state.
Episode	One full attempt, from the start until the agent wins or loses.

RUN IT — LESSON 1

`python 01_explore_environment.py` resets the world, takes a few moves, and prints the state, reward, and "done" flag each time — so these five words stop being abstract.

3 Types of reinforcement learning

RL methods are usually sorted along two questions. First: does the agent build a *model* of how the world works? Second: does it learn from its *own* actions or from someone else's?

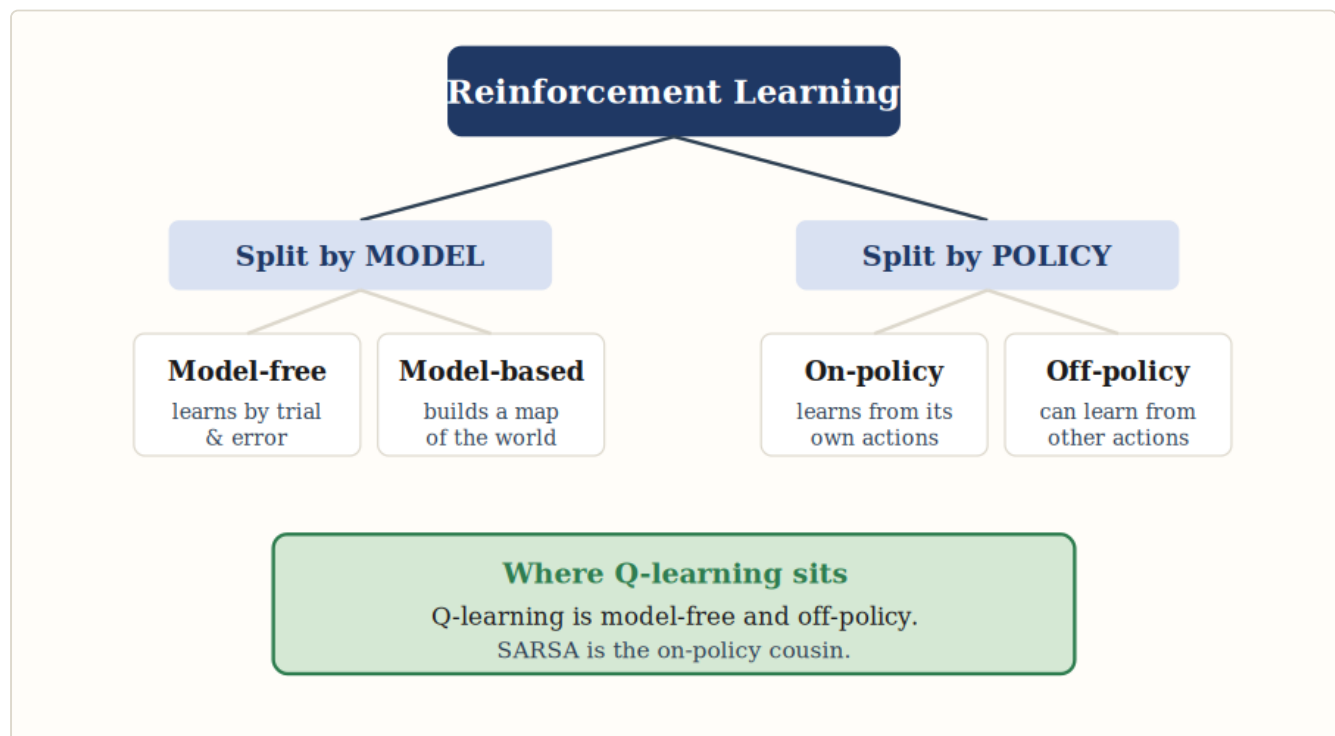


Figure 2 — Two ways to categorise RL methods, and where Q-learning fits.

- **Model-free** methods learn purely by trial and error. They do not try to predict the world; they just remember what worked.

- **Model-based** methods build an internal map of the world (the odds of moving from one state to another) and plan against it. More powerful, but they need more memory.
- **On-policy** methods learn from the actions they actually take. **Off-policy** methods can learn the best strategy even while exploring — the policy that *generates* the moves (the behavior policy) is allowed to differ from the policy being *improved* (the target policy). Q-learning does this: its update always looks at the best next move, no matter what exploratory move was actually played.

WHERE Q-LEARNING SITS

Q-learning is **model-free** and **off-policy**. Its close relative **SARSA** is on-policy. Both come from the same family.

4 The Markov Decision Process (MDP)

The **Markov Decision Process** is the formal name for the whole setup: an agent observing an environment, receiving a reward and a next state, and acting on it. Written out, it is the tuple (**S** states, **A** actions, **P** transition probabilities, **R** rewards, γ discount). It is the frame that turns "an agent learning by trial and error" into a precise problem we can solve.

The one property that makes it "Markov" is this: **the next state depends only on the current state and the action taken — not on the entire history** of how the agent got there. That is why the state must capture everything relevant. If it does, the agent never needs to remember the past.

IN PLAIN TERMS

"Markov" just means: *where you are now, plus what you do next, tells you everything you need; the path that got you here doesn't add anything*. A chess position is Markov — you can play on perfectly without knowing the move order that produced it.

5 Our practice world: FrozenLake

All the code in this guide uses one small, friendly environment so you can focus on ideas rather than setup.

FrozenLake is a 4×4 grid. You start top-left and try to reach the goal bottom-right without falling in a hole. Reaching the goal gives a reward of 1; everything else gives 0.

FrozenLake: a 4 x 4 grid world

START #0	ice #1	ice #2	ice #3
ice #4	HOLE #5	ice #6	HOLE #7
ice #8	ice #9	ice #10	HOLE #11
HOLE #12	ice #13	ice #14	GOAL #15

Start at #0, reach the GOAL at #15. Step on a HOLE and the episode ends.
16 states, 4 actions (up/down/left/right) → a 16 x 4 Q-table.

Figure 3 — FrozenLake: 16 tiles (states) and 4 moves (actions).

Because there are 16 states and 4 actions, the agent's knowledge fits neatly in a 16×4 grid of numbers — the **Q-table**, which the next section explains.

ONE SETTING MATTERS

By default FrozenLake is **slippery**: the ice is stochastic, so an intended move can slide the agent sideways. That is realistic but confusing when you are starting out, and it lowers the best achievable success rate. Every lesson in this guide creates the world with `is_slippery=False` so the ice behaves predictably and a well-trained agent can reach the goal almost every time.

RUN IT — LESSON 2

`python 02_random_agent.py` lets a purely random walker try 1000 times. With this guide's settings (non-slippery FrozenLake-v1, 1000 episodes) it reaches the goal only about **2%** of the time — an illustrative figure that varies a little run to run. That is our *baseline*: the number a smart agent must beat.

6 Q-learning and the Q-table

Q-learning gives each pair of (state, action) a score called a **Q-value** — think of it as "how good is this move from this tile?" All the scores together make the **Q-table**. At the start every score is zero: the agent knows nothing. Through trial and error it fills the table in.

The Q-table: a score for every (state, action)				
state \ action	← Left	↓ Down	→ Right	↑ Up
state 0	0.02	0.51	0.03	0.01
state 5	0.00	0.00	0.00	0.00
state 14	0.10	0.62	0.11	0.09

green = best action in that row

To act, read the agent's current state row and pick the highest number.
All zeros at the start → the agent fills these in by trial and error.

Figure 4 — The Q-table gives every (state, action) a score. To act, read your row and pick the biggest number.

To choose a move, the agent looks up its current state (a row) and picks the action with the highest score. To *learn*, it adjusts that score after every move using a single formula.

The one formula (the Q-learning rule)

The Q-learning update rule, in plain parts

$$Q(s, a) \leftarrow Q(s, a) + \alpha (r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a))$$

α = learning rate
how big a step

r = reward now
just received

γ = discount
value of the future

$\max Q(s', \cdot)$
best next move

Read it as: nudge the old score toward "reward now + how good the next state can be."
The part in parentheses is the "surprise" — how wrong the old guess was.

Figure 5 — The Q-learning update rule, broken into plain-language parts.

After each move the agent nudges the old score a little toward "the reward it just got, plus how good the best next move looks." The size of the nudge is set by the **learning rate** (α), and how much the future counts is set by the **discount factor** (γ).

EXPLORE VS EXPLOIT

Early on the agent should **explore** (try random moves to discover the world). Later it should **exploit** (trust its table). We blend the two with *epsilon-greedy*: act randomly with probability epsilon, which shrinks over time.

The heart of the code

```
# The Q-table starts as all zeros: the agent knows nothing.
Q = np.zeros((n_states, n_actions))

# ...inside the loop, after taking an action...
best_next = np.max(Q[new_state, :])
Q[state, action] = Q[state, action] + lr * (
    reward + gamma * best_next - Q[state, action]
)
```

RUN IT — LESSON 3 (THE MAIN EVENT)

`python 03_qlearning_frozenlake.py` trains the agent, prints the filling Q-table, draws the learned route through the grid, and reports a **near-100% success rate** — a huge jump over the 2% baseline.

Watching it learn

Progress is easier to feel as a picture. The chart below (produced by Lesson 4) shows the success rate climbing from near zero to around 90% as training goes on.

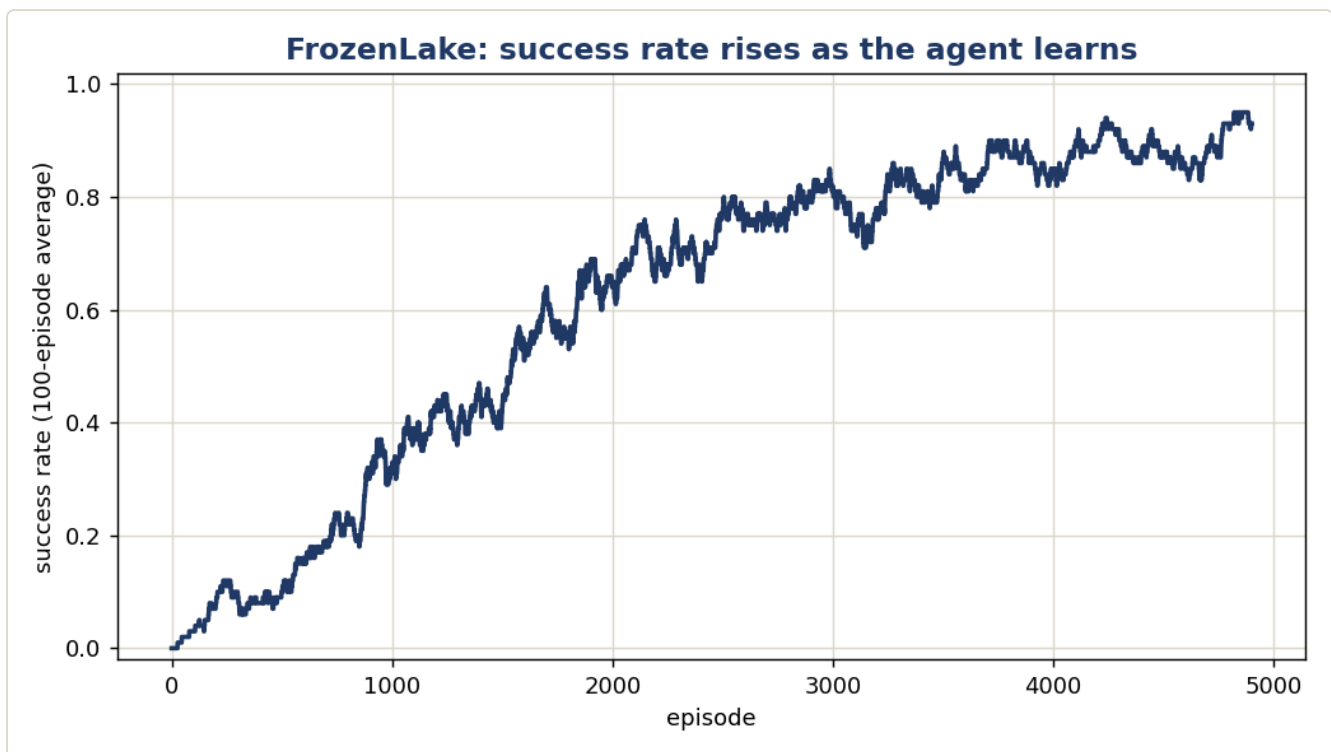


Figure 6 — The agent's success rate rising over 5000 episodes (from Lesson 4).

RUN IT — LESSON 4

```
python 04_visualize_learning.py
```

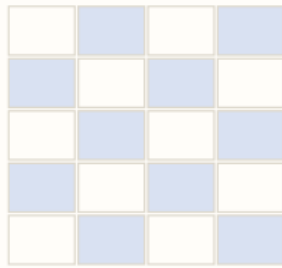
 trains the agent and saves this exact style of chart as `learning_curve.png` .

7 When the table gets too big: Deep Q-learning

A 16×4 table is tiny. But real problems can have millions of states — far too many to store. **Deep Q-learning** replaces the table with a small **neural network** that *estimates* a Q-value for any state it is shown, letting the agent generalize across large state spaces. It is the same Q-learning idea, with a bigger toolbox.

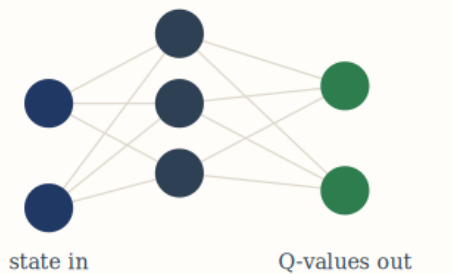
When the table gets too big: Deep Q-learning

Small world → Q-table



one row per state
works when states are few

Huge world → neural network



state in
estimates Q without storing every cell

Q-values out

Same idea, bigger toolbox: the network replaces a table too large to hold.

Figure 7 — For large worlds, a neural network replaces a Q-table too big to store.

The Keras model below illustrates the core piece: *Q-value prediction*. A full DQN training system normally adds two stabilizers on top — an **experience replay buffer** (storing past transitions to sample from) and a **target network** (a slowly-updated copy used for the learning target). The model on its own is the function approximator, not the whole system.

The course transcript builds this network with **Keras**, stacking **Dense** layers inside a **Sequential** model. The hidden layers use the **ReLU** activation; the final layer uses a **linear** activation, because each output is a Q-value — an estimate of expected discounted return, not a normalized probability. (When observations are images, convolutional layers are used instead of Dense; the architecture follows the input type.)

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(1, STATE_SIZE)),
    layers.Flatten(),
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(16, activation="relu"),
    layers.Dense(NUM_ACTIONS, activation="linear"),
])
```

RUN IT — LESSON 5 (OPTIONAL)

`python 05_build_dqn_model.py` builds this network and prints its summary. For the CartPole example it uses (a 4-number state and 2 actions) the network has **658** tunable parameters — matching the transcript. If TensorFlow is not installed, the script still runs and explains the model in words.

8 Installing the tools

Two libraries do most of the work in RL code: an **environment** library (Gymnasium, the maintained successor to OpenAI Gym) and, for deep methods, a **neural-network** library (TensorFlow/Keras).

Modern TensorFlow has simplified installation — you no longer choose between separate CPU and GPU packages. The single `tensorflow` package is the right starting point:

Goal	Command	Notes
CPU (all platforms)	<code>pip install tensorflow</code>	The right choice for learning. Works on Linux, macOS, and Windows.
GPU (Linux / WSL2)	<code>pip install tensorflow[and-cuda]</code>	Only if you have a supported NVIDIA GPU and want acceleration.

GOOD TO KNOW

Official GPU support is not provided for macOS, and native-Windows GPU support exists only in older TensorFlow versions (use WSL2 for GPU on Windows). You do not need a GPU for anything in this guide — the CPU package is plenty.

YOU MAY ALSO SEE OLDER NAMES

Legacy tutorials mention `tensorflow-gpu` and nightly builds like `tf-nightly`. The separate `tensorflow-gpu` package is retired (GPU support is now built into `tensorflow`), and "nightly" builds are bleeding-edge previews — avoid them while learning.

USE A VIRTUAL ENVIRONMENT

Always install into a *virtual environment* — a private sandbox for this project — so its packages never collide with the rest of your system. The companion `README.md` gives the exact commands for macOS, Linux, and Windows.

9 The companion code package

The file `RL_Sample_Programs.zip` contains five short, heavily-commented lessons that share one environment. Run them one at a time to read along, or run them all at once to check your setup.

File	What it teaches
<code>common.py</code>	Shared FrozenLake setup used by every lesson.
<code>01_explore_environment.py</code>	State, action, reward, step, episode — in action.
<code>02_random_agent.py</code>	A random baseline to beat (~2%).
<code>03_qlearning_frozenlake.py</code>	Q-learning and the Q-table — the core lesson.

04_visualize_learning.py	A chart of the agent improving over time.
05_build_dqn_model.py	A Keras Deep-Q network (optional).
run_all.py	Runs and verifies every lesson at once.
requirements.txt	The small list of packages to install.
README.md	Step-by-step virtual-environment setup.

Three commands to get going

```
# 1. create and activate a virtual environment (macOS/Linux)
python3 -m venv venv && source venv/bin/activate

# 2. install the requirements
pip install -r requirements.txt

# 3. run everything, or one lesson at a time
python run_all.py
```

10 Quick reference

The loop

state → action → reward + next state, repeat.

Goal

Maximise total reward over an episode.

Q-value

A score for one (state, action) pair.

Q-table

All Q-values: states × actions.

Update rule

$Q \leftarrow Q + \alpha(r + \gamma \cdot \max_{Q'} Q - Q)$.

α learning rate

How big each learning step is.

γ discount

How much future reward counts.

Epsilon-greedy

Explore sometimes, exploit the rest.

Model-free

Learn by trial and error (Q-learning).

Off-policy

Learn the best path while exploring.

Deep Q-learning

A network replaces a too-big table.

Sequential + Dense

Keras building blocks for that network.

11 Vocabulary & concepts

Every term below appears in this guide or the course it is built from. Use it as a lookup while you read and code.

Term	Definition
Action (A)	All the possible moves an agent can take in an environment. Denoted by the value A. Each action affects the environment and moves the agent toward a new state.
Activation Function	A function attached to a neural-network layer that determines its output. In the course examples, ReLU is used for hidden layers and linear for the final output layer.
Adam	An optimizer imported from <code>keras.optimizers</code> , used to train the neural-network model.
Agent	A hypothetical entity that performs actions in an environment to gain a reward. It is the learner and decision-maker that observes state, acts, receives rewards, and develops future strategies.
argmax (tf.argmax / np.argmax)	A function that returns the index of the maximum value. Used to select the best action from the Q-values (e.g., <code>np.argmax(Q[s,:])</code>).
Asynchronous Dynamic Programming	An approach to which Q-learning is related; Q-learning can be viewed as an asynchronous dynamic programming method.
BoltzmannQPolicy	[Legacy - Keras-RL] A policy from the older <code>keras-rl</code> library, used with its DQN agent to select actions probabilistically. <code>keras-rl</code> is unmaintained; the companion code does not use it.
CartPole-v0	A built-in OpenAI Gym environment used in the Keras examples for building and testing a model.
CNN (Convolutional Neural Network)	A neural-network architecture suited to image-shaped inputs. It is used inside a Deep Q-Network only when the observations are images; vector or discrete states use ordinary Dense (fully-connected) layers instead. DQN is not inherently convolutional.
CPU / GPU Support	TensorFlow packages differ by hardware target. CPU-only builds run without a graphics card; GPU builds add acceleration via a supported graphics processor.
Deep Q-learning (Deep Q-Network / DQN)	A value-based method that combines deep learning and reinforcement learning. It uses a neural network in place of a Q-table to estimate Q-values, letting an agent generalize across large state spaces. The network architecture depends on the observation type (Dense layers for vector/discrete states; convolutional layers when observations are images). A full DQN training system also typically adds two stabilizers: an experience replay buffer and a periodically updated target network.
Dense	A Keras layer (<code>keras.layers.Dense</code>) implementing a fully connected layer. Added to the Sequential model with a specified number of units (e.g., <code>Dense(16)</code>).

Term	Definition
Discount Factor (gamma)	A factor between 0 and 1 (e.g., 0.95) that weights future rewards relative to immediate ones. Part of the Markov Reward Process tuple and the Q-learning update rule. Written as the Greek letter gamma in the formulas; the course's Python code happens to name the variable <code>y</code> .
Discounted Reward	The value given for future references — the portion of a Q-value that accounts for rewards expected later, scaled by the discount factor.
DQNAgent	[Legacy - Keras-RL] A pre-built deep Q-network agent from the older keras-rl library. keras-rl no longer runs on current Python; modern projects build the agent directly in Keras or use a maintained library such as Stable-Baselines3.
Environment	The scenario or world the agent must face. It receives the agent's action, returns a new state and a reward, and defines the dynamics the agent learns.
Episode	One complete run through the environment from reset to termination. The training loop runs a set number of episodes (e.g., 2000) to accumulate score.
Experience Tuple	The set (s, a, r, s') — state, action, reward, next state — that captures a single learning transition used to update the Q-value.
Finite Markov Decision Process (Finite MDP)	An MDP with a finite set of states and actions; one of the three process elements involved in Q-learning.
Flatten	A Keras layer that reshapes the input (e.g., the environment's observation shape) into a flat vector as the model's input layer.
FrozenLake-v0	A built-in OpenAI Gym environment organized as a 4x4 grid of blocks — each a start block, gold block, safe frozen block, or dangerous hole. Used for the Q-learning examples.
GradientDescentOptimizer	[Legacy - TensorFlow 1.x] The old gradient-descent trainer. In modern Keras use <code>keras.optimizers.SGD</code> (plain gradient descent) or <code>keras.optimizers.Adam</code> .
Immediate Reward	The point given when an agent moves from the current state to another while taking an action — the near-term component of a Q-value.
Keras	A high-level neural-network library used to build models for reinforcement learning (Sequential models, Dense/Activation/Flatten layers, optimizers).
Learning Rate (lr / α)	A parameter (e.g., <code>lr = 0.8</code>) controlling how much new information overrides old Q-values in the update rule.
Linear Activation	The activation function used on the final output layer of the model (as opposed to ReLU on hidden layers).
Loss Function	A measure of model error, computed as the sum of squared differences between the target Q-values (<code>nextQ</code>) and predicted Q-values (<code>Qout</code>).

Term	Definition
Markov Decision Process (MDP)	The process of an agent observing the environment, receiving output consisting of a reward and the next state, and then acting upon it. A classic approach to identifying the problem of learning from interactions to achieve goals. Formally the tuple (S, A, P, R, γ) : states, actions, transition probabilities, reward function, and discount factor.
Markov Property	The condition that the next state depends only on the current state and the action taken, not on the full history: $P(S_{t+1} S_t, A_t) = P(S_{t+1} S_1, A_1, \dots, S_t, A_t)$. The current state plus action captures all relevant information from the past.
Markov Reward Process (MRP)	A random process used for decision making, derived from accumulated reward over sample sequences. Represented by the tuple (S, P, R, γ) : finite state space, state transition probability function, reward function, and discount factor.
matmul (tf.matmul)	TensorFlow matrix multiplication, used to multiply inputs by weights (W) to produce the Q-value output matrix (Q_{out}).
Model	A simulation of the dynamics of the environment in which the agent operates, used to produce different outcomes.
Model-based Reinforcement Learning	An approach in which the model learns the transition probability from state-action pairs to next states, building a representation of the world. May require extra space to store states and actions, which can make it impractical for large problems.
Model-free Reinforcement Learning	An approach that learns what to do directly from experience without building an explicit model of the environment's transitions and rewards. Model-free does not mean memory-free: tabular Q-learning stores one value per state-action pair, and deep versions hold network parameters and replay buffers. Q-learning is the classic example.
NumPy (np)	A Python package providing number manipulation and array capabilities, used for the Q-table and random number generation.
Off-policy	A learning setting where the policy that generates the data (the behavior policy) differs from the policy being evaluated and improved (the target policy). Q-learning is off-policy: its update uses the greedy best next action for the target, regardless of the possibly exploratory action actually taken.
On-policy	A learning setting where the agent learns values based on its current action, derived from its current policy.
OpenAI Gym (gym)	A framework provided by OpenAI supplying built-in environments (e.g., CartPole-v0, FrozenLake-v0) for reinforcement learning.
Optimal Action Policy	The best action-selection strategy; one of the three process elements involved in Q-learning, which learns the optimum policy even when actions are selected by a more exploratory or random policy.

Term	Definition
Optimizer	A component that adjusts model parameters to minimize loss (e.g., Adam in Keras; SGD or Adam in modern TensorFlow).
Policy (π)	The strategy an agent applies to determine the next action based on the current state. Denoted by the pi symbol.
Policy Gradient	A policy-based method that determines outcomes based on policy and can learn from the probability generated for every action.
Q-learning	A model-free reinforcement learning technique and off-policy temporal- difference algorithm. Provides an agent the capability of learning to act optimally in Markovian domains by estimating the values of state- action pairs. Tabular Q-learning is guaranteed to converge to the optimal action-value function under conditions such as a finite MDP, sufficient exploration of every state-action pair, and an appropriately decaying learning rate. These guarantees do not automatically transfer to neural- network (function-approximation) versions.
Q-learning Rule	The update formula: $Q(s,a) := Q(s,a) + \alpha (r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a))$ revising the Q-value using the immediate reward, discounted best future value, and learning rate.
Q-table	A lookup table storing a Q-value for every state-action pair, initialized to zeros with <code>np.zeros([env.observation_space.n, env.action_space.n])</code> .
Q-value (Q)	Similar to value but taking an extra parameter — the current action. Represented by $Q[s,a]$ and equal to the immediate reward plus the discounted future reward. Denoted by the value Q.
ReLU (Rectified Linear Unit)	The activation function used for the hidden Dense layers of the model.
Reinforcement Learning (RL)	A method of machine learning in which a software agent learns to perform actions in an environment that lead to maximum reward, through repeated trials of exploring and exploiting learned knowledge. Used when an agent lacks the labeled feedback of supervised learning.
Reward (R)	The immediate return sent back from the environment to evaluate the last action taken by the agent. Denoted by the value R. Agents are incentivized to maximize cumulative reward.
SARSA (State-Action-Reward-State-Action)	An on-policy algorithm derived from the MDP policy. Named for the sequence its Q-value update depends on: current state (S), action (A), reward (R), next state (S), and next action (A2).
Score Over Time (Scoreovertime)	A performance measure computed as the sum of rewards (rList) divided by the number of episodes.
Sequential	A Keras model class (<code>keras.models.Sequential</code>) that builds a network as a linear stack of layers.

Term	Definition
SequentialMemory	[Legacy - Keras-RL] The experience-replay buffer from the older keras-rl library. The same idea - storing past transitions to sample during training - is central to modern DQN, but under different APIs.
State (S)	The current situation returned by the environment. Denoted by the value S. Should capture all relevant information from history (Markov property).
State Transition Probability Function (P)	The function giving the probability of moving from one state to the next, part of the Markov (Reward) Process tuple.
Supervised Learning	A machine learning approach in which agents are given labeled information/ feedback for decision making — contrasted with RL, which learns from rewards without such feedback.
TensorFlow (tf)	A machine learning framework used as a backend for building and training reinforcement learning models. Installed via pip.
TensorFlow Packages	Distribution variants selected by need: - tensorflow : stable, CPU-only (Ubuntu/ Windows) - tensorflow-gpu : stable, with GPU support - tf-nightly : preview nightly build, CPU only - tf-nightly-gpu : nightly build with GPU support
Temporal Difference Learning	The class of methods, used by Q-learning, that updates value estimates based on other learned estimates (bootstrapping) rather than waiting for a final outcome.
tf.placeholder	[Legacy - TensorFlow 1.x] An input slot fed at session-run time in the old graph API. In modern TensorFlow 2 / Keras you use keras.Input and ordinary Python values instead.
tf.Variable	A TensorFlow container for trainable values such as weights and biases. Still valid in TF2, but you rarely create them by hand - Keras layers manage their own variables.
Transition Probability	The probability, learned in model-based RL, of moving from a given state-action pair to the next state.
Value (V)	The expected long-term return from a state, including a discount factor. Denoted by the value V. Differs from Q-value in that it does not take the action as a parameter.
Value-based Method	An approach (e.g., Deep Q-learning) that derives decisions from estimated values, as opposed to policy-based methods.
Virtual Environment (virtualenv)	An isolated Python environment into which TensorFlow can optionally be installed, as an alternative to a system-wide install.
Weights (W)	Trainable TensorFlow variables multiplied with the input to produce Q-value outputs; initialized with random uniform values.

12 Learn more

These sources go deeper than this beginner's guide when you are ready.

[Sutton & Barto — Reinforcement Learning: An Introduction](#)

The standard free textbook. The definitive reference for MDPs, Q-learning, and convergence conditions.

<http://incompleteideas.net/book/the-book-2nd.html>

[Gymnasium \(maintained OpenAI Gym\)](#)

The current, actively maintained home of the Gym environments used in this guide.

<https://gymnasium.farama.org/>

[Gymnasium — FrozenLake environment](#)

Full description of the FrozenLake world the sample programs use, including the `is_slippery` setting.

https://gymnasium.farama.org/environments/toy_text/frozen_lake/

[Keras — Getting started](#)

Official Keras docs: Sequential models, Dense layers, activations, optimizers.

https://keras.io/getting_started/

[TensorFlow — Install guide](#)

Current official installation guidance for CPU and GPU.

<https://www.tensorflow.org/install>

[Mnih et al. \(2015\) — Human-level control through deep RL](#)

The original DQN paper, describing experience replay and the target network.

<https://www.nature.com/articles/nature14236>

Reinforcement Learning Tools & Frameworks

A Beginner's Qwik Guide · AI Foundations Series

© 2026 David Grunwald. All rights reserved.

This training guide and its companion code package are provided for educational use.